



华南师范大学

本科学生实验（实践）报告

院 系： 计算机学院

实验课程： 编译原理

实验项目： LALR(1)分析生成器

指导老师： 黄煜廉

开课时间： 2023 ~ 2024 年度第 二 学期

专 业： 计算机科学与技术

班 级： 2022级计科2班

学生姓名： 黄兆清

华南师范大学教务处

华南师范大学实验报告

学生姓名 黄兆清 学 号 20222131044
专 业 计算机科学与技术 年级、班级 2022级计科2班
课程名称 编译原理 实验项目 LALR(1)分析生成器
实验类型 ☐验证 ☐设计 ☐综合 实验时间 2024 年 6 月 18 日
实验指导老师 黄煜康 实验评分 _____

一、实验内容：

设计一个应用软件，以实现 LALR(1)分析生成器。

1. 提供一个文法输入编辑界面，让用户输入文法规则(可保存、打开存有文法规则的文件)。
2. 计算文法各非终结符号的 first 集合与 follow 集合，并提供窗口让用户查看这些集合结果(采用表格形式呈现)。
3. 提供窗口让用户查看文法对应的 LR(1)DFA 图(可以用画图或表格方式展示图的节点和边数据)。
4. 提供窗口让用户查看文法对应的 LALR(1)DFA 图(可以用画图或表格方式展示图的节点和边数据)。
5. 提供窗口让用户查看文法对应的 LALR(1)分析表(如果文法是 LALR(1)文法的话，采用表格形式呈现)。
6. 完善的软件文档，包括使用说明和技术文档。
7. Windows 界面的应用程序。

二、实验目的：

- 通过实验掌握如何设计和实现 LALR(1)分析生成器，从而深入理解和应用 LR 分析算法。
- 通过实验理解 LR(1)语法分析的基本原理和实现过程，包括 LR(1)项集的构造、状态转移图的生成以及分析表的生成方法。
- 通过实验提升对 LR(1)分析算法的理解和掌握，加深对编译器前端工作的认识。
- 通过实验将学习到的理论知识应用到实际项目中，培养解决复杂问题和实现算法的能力。
- 通过实验掌握软件设计与开发的基本方法，包括界面设计、数据结构选择和算法实现等方面的技能。
- 通过实验在设计和实现过程中遇到的挑战和问题，提升问题解决能力和调试技巧。
- 通过实验完善软件文档，包括详细的使用说明和技术文档，培养书面表达和文档编写能力。

三、实验文档

1. 实验项目概述

- 提供一个文法输入编辑界面,让用户输入文法规则(可保存、打开存有文法规则的文件)。
- 计算文法各非终结符号的 first 集合与 follow 集合,并提供窗口让用户查看这些集合结果(采用表格形式呈现)。
- 提供窗口让用户查看文法对应的 LR(1)DFA 图(可以用画图或表格方式展示图的节点和边数据)。
- 提供窗口让用户查看文法对应的 LALR(1)DFA 图(可以用画图或表格方式展示图的节点和边数据)。
- 提供窗口让用户查看文法对应的 LALR(1)分析表(如果文法是 LALR(1)文法的话,采用表格形式呈现)。
- 完善的软件文档,包括使用说明和技术文档。
- Windows 界面的应用程序。

2. 实验项目设计

- 文法输入编辑界面
 - 功能描述: 提供一个用户界面,允许用户输入和编辑文法规则。
 - 需求细节:
 - 用户能够直接在界面上输入文法规则。
 - 提供保存和打开已保存文法规则的功能,允许用户管理和维护多个文法规则文件。
- 计算文法的 first 集合与 follow 集合
 - 功能描述: 根据用户输入的文法规则,计算并展示各非终结符号的 first 集合和 follow 集合。
 - 需求细节:
 - 算法正确性和效率: 确保计算过程准确无误,并在合理时间内

完成计算。

- 结果展示：以表格形式呈现计算结果，方便用户查看和分析。

- LR(1)DFA 图展示

- 功能描述：根据用户输入的文法规则，生成 LR(1)文法对应的 DFA 图，并提供可视化界面展示图的节点和边数据。
- 需求细节：
 - 可视化展示：支持图形化展示和表格展示两种方式，用户可选择查看。
 - 交互性：用户可以通过界面操作，缩放、导航和查看具体的节点和边信息。

- LALR(1)DFA 图展示

- 功能描述：生成 LALR(1)文法对应的 DFA 图，并提供类似 LR(1)DFA 图的可视化展示。
- 需求细节：
 - 同 LR(1)DFA 图展示需求，支持图形化和表格化两种展示方式。

- LALR(1)分析表展示

- 功能描述：如果输入文法是 LALR(1)文法，生成并展示 LALR(1)分析表。
- 需求细节：
 - 表格展示：以表格形式呈现分析表，包括状态、动作和转移信息。
 - 分析表的正确性：确保生成的分析表符合 LALR(1)文法的要求。

- 界面要求

- 功能描述：应用程序的操作界面应为 Windows 界面，友好且易于使用。
- 需求细节：
 - 界面设计：符合 Windows 应用程序设计规范，包括菜单、工具栏、状态栏等标准元素。
 - 用户体验：提供直观的操作流程和反馈，减少用户的学习成本和操作复杂度。

3. 数据结构设计

- 数据结构 Grammar :
 - Grammar 结构体包含两成员 :
 - sign : 表示文法符号的字符串。
 - grammar : 表示与该符号相关联的语法规则的字符串向量。
 - 重载了 == 运算符 , 用于比较两个 Grammar 对象是否相等。相等的条件是 sign 相等且 grammar 向量相等。
- IndexedSet<T> 的数据结构 :
 - 一个存储元素的向量 (elements) 和一个通过哈希映射实现的元素到索引的快速查找机制 (elementIndexMap)。
 - 数据结构详解 :
 - 元素存储向量 (elements) :
 - 类型 : std::vector<T>
 - 作用 : 存储所有插入到集合中的元素。
 - 特性 :
 - 元素按照插入顺序存储 , 支持通过索引访问和顺序遍历。
 - 支持动态扩展和收缩 , 根据需要自动调整存储空间。
 - 元素到索引的哈希映射 (elementIndexMap) :
 - 类型 : std::unordered_map<T, size_t>
 - 作用 : 将集合中的每个元素 T 映射到其在 elements 向量中的索引 size_t。
 - 特性 :
 - 哈希表保证了在平均情况下 (均摊时间复杂度 $O(1)$) 的快速查找、插入和删除操作。
 - 提供了高效的元素查找功能 , 避免了线性搜索的低效率问题 , 特别是在大数据量时。
 - 操作实现 :
 - 插入操作 (insert(const T &element)) :
 - 首先通过 elementIndexMap 查找元素是否已存在 , 如果存在则返回其索引 ; 否则 , 在 elements 向量末尾添加元素 , 并

更新 elementIndexMap 中的映射关系。

- 获取元素操作 (getElement(size_t index)) :
 - 直接通过索引访问 elements 向量获取元素。
- 删除元素操作 (remove(const T &element)) :
 - 首先在 elementIndexMap 中查找元素，找到后获取其索引并从 elements 中移除。
 - 如果被移除元素不在向量末尾，将末尾元素移动到被移除位置以保持向量连续性，并更新其在 elementIndexMap 中的映射关系。
- 查找元素操作 (find(const T &element)) :
 - 通过 elementIndexMap 查找元素，并返回其索引，如果不存在则返回特定值（这里返回 -1，但应返回 size_t 类型的特殊值表示未找到）。
- 其他操作 :
 - 支持合并元素、清除集合、检查集合是否为空等操作，通过组合 std::vector 和 std::unordered_map 提供了丰富的集合管理功能。

◦

◦ 枚举 _op :

- 定义了三个枚举值 s, j, g，用于表示操作类型。

◦

◦ 结构体 operation :

▪ 成员变量 :

- op : 枚举 _op 类型，表示某种操作。
- n : 整数，表示状态编号。
- next_sign : IndexedSet<string>，存储一组字符串，表示规约操作的超前查看符号。

◦

◦ 结构体 Item :

▪ 成员变量 :

- `grammar`：整数，表示语法规则编号。
- `dot`：整数，表示文法当前进行位置。
- `is_epsilon`：布尔值，表示是否为空串。
- `next_sign`：`IndexedSet<string>`，存储一组字符串，表示规约操作的超前查看符号。

▪ 运算符重载：

- `operator==`：用于比较两个 `Item` 结构是否相等。

○

○ 结构体 `Statu`：

▪ 成员变量：

- `items`：`IndexedSet<Item>`，存储一组 `Item`，表示状态集合。

▪ 运算符重载：

- `operator==`：用于比较两个 `Statu` 结构是否相等。

○

○ 类 `LR1`

▪ 成员变量：

- `SetsSize` 和 `SignSize`：分别表示状态集合和符号集合的预设大小。
- `status`：`IndexedSet<Statu>`，存储所有的状态集合。
- `table`：`unordered_map<pair<int, int>, vector<operation>, hash<pair<int, int>>>>`，存储 LR(1) 分析表，使用状态和符号的组合作为键，存储操作列表。
- `NTsigns` 和 `signs`：分别是非终结符和符号的 `IndexedSet<string>`，用于存储文法中的非终结符和所有符号。
- `grammers`：`IndexedSet<Grammar>`，存储文法的所有产生式。
- `FirstSet`：`map<string, vector<string>>`，存储文法符号的 First 集合。

▪ 内部函数：

- `inputGrammers`：将输入的文法规则存储到 `grammers` 中，并更新非终结符集合 `NTsigns` 和符号集合 `signs`。

- `getNewItems`：根据非终结符获取新的项目集合。
- `closure`：计算项目集的闭包操作，根据文法产生式展开项目。
- `calNextSign`：计算项目集中每个 item 的超前查看符号。
- `work`：主要函数，生成 LR(1) 分析表和状态机。

○ 类 LALR1：

- 成员变量：
 - `SetsSize` 和 `SignSize`：状态集合和符号集合的预设大小。
 - `status`：IndexedSet<Statu>，存储所有的状态集合。
 - `table`：unordered_map<pair<int, int>, vector<operation>, hash<pair<int, int>>>>，LR(1) 分析表。
 - `NTsigns` 和 `signs`：IndexedSet<string>，非终结符和符号集合。
 - `grammers`：IndexedSet<Grammer>，文法产生式集合。
 - `FirstSet`：map<string, vector<string>>，文法符号的 First 集合。
- 函数成员：
 - 构造函数 `LALR1()`：调用父类的构造函数，初始化所有成员变量。
 - `mergeStates()`：合并具有相同核心项的状态，生成 LALR(1) 状态集合。
 - `work()`：重载了父类的 `work()` 方法，首先调用父类的方法生成 LR(1) 状态集合，然后调用 `mergeStates()` 方法生成 LALR(1) 状态集合，并最终调用 `rebuildTable()` 方法重新生成操作表。
 - `rebuildTable()`：重新构建 LALR(1) 分析表，处理合并后的状态集合。
 - `findStateWithItem(const Item &item)`：根据给定的项目，查找并返回包含该项目的状态的索引。
- 辅助函数：
 - `printStateTable(const std::string &filename)`：生成状态表的可视化表示，输出为 DOT 文件格式。该函数输出状态表格，显示每个状态对应的操作内容（移进或规约）。

4. 关键算法设计

- First_Follow 类生成 First_Follow 集合的算法：
 - 初始化阶段 (scan() 方法)：
 - 处理输入的文法，将每个文法规则分解为左部和右部的符号序列，并初始化 First、Follow 和 nullable。
 - 生成 First 和 Follow 集合 (gen() 方法)：
 - 首先初始化终结符的 First 集合为自身。
 - 使用循环迭代计算每个非终结符的 First 集合，考虑到文法的可空性。
 - 对于每个文法符号，根据其后续符号的可空性来更新对应的 Follow 集合。
 - 使用 haveChange 标志来判断集合是否发生变化，直到没有变化为止，确保计算得到的 First 和 Follow 集合稳定。
- LR1 类中的 work 函数的算法：
 - 初始化：
 - 创建一个栈 notRead 用于存储待处理的状态编号。
 - 创建起始状态 startStatu，初始化一个起始项 startItem，并进行闭包运算和计算后继查看符号。
 - 状态处理循环：
 - 使用栈 notRead 来迭代处理所有待处理的状态。
 - 对当前栈顶状态进行处理，获取其包含的 LR(1)项集合 this_statu。
 - 状态转移和操作填充：
 - 对于每个 LR(1)项，根据点的位置判断是进行移入操作还是归约操作：
 - 若点在产生式末尾，执行归约操作，并在分析表中记录对应的归约动作。
 - 若点不在产生式末尾，执行移入操作，计算下一个状态的 LR(1)项集合，并在分析表中记录移入操作。
 - 状态集合管理：

- 对于新生成的状态，进行闭包运算，并将新状态加入状态集合中。
- 若状态集合中已存在相同的状态，则不添加。
- 循环终止条件：
 - 所有状态都处理完成后，栈 notRead 为空，循环结束。
- 详细步骤说明：
 - 闭包运算 (closure 函数)：
 - 对于状态集合中的每个 LR(1)项，通过查找产生式右部的非终结符，将其后继的 LR(1)项加入集合。
 - 如果新加入了 LR(1)项，则标记 have_change 为 true，继续循环直到没有新项加入。
 - 计算超前查看符号 (calNextSign 函数)：
 - 根据当前状态中的每个 LR(1)项，找出它的后继符号，计算其后继查看符号集合。根据文法产生式中的符号和 First 集合进行计算。
 - 如果当前项是空串，则传递空串信息，影响到状态转移时的计算。
 - 状态转移和分析表填充：
 - 遍历当前状态集合中的每个 LR(1)项，根据点的位置判断执行移入操作还是归约操作。
 - 移入操作：计算下一个状态的 LR(1)项集合，填充到分析表中，并将新状态加入状态集合。
 - 归约操作：根据归约的文法产生式，填充对应的归约操作到分析表中。
 - 分析表 (table)的结构：
 - 使用 unordered_map<pair<int, int>, vector<operation>> table 来表示分析表，其中键是当前状态和当前输入符号的索引对，值是操作类型(_op)和对应的状态编号
 - LALR1 类中 mergeStates 函数的算法：
 - 核心项的定义：

- 核心项是 LR(1)项中除去后继查看符号的部分，用于判断两个状态是否具有相同的核心项。
- 状态分类和合并：
 - 使用 `unordered_map<IndexedSet<Item>, Status>` `coreToStateMap` 来存储每个核心项集合对应的状态。
 - 遍历当前的状态集合 `status`，对每个状态进行处理：
 - 提取状态中所有 LR(1)项的核心项集合。
 - 根据核心项集合判断是否已存在相同核心项集合的状态：
 - 如果已存在，将当前状态中的所有 LR(1)项添加到已存在状态的项集中。
 - 如果不存在，将当前状态作为新的状态加入到 `coreToStateMap` 中。
- 状态集合的更新：
 - 根据 `coreToStateMap` 中的结果重新构建状态集合 `status`，确保合并后的状态集合中不包含重复的状态。
- 详细步骤说明：
 - 核心项的提取：
 - 对于每个状态，遍历其 LR(1)项集合，将每个 LR(1)项的后继查看符号清空，得到核心项集合。
 - 状态的合并：
 - 将相同核心项集合的状态合并为一个新的状态。对于每个核心项集合，判断是否已存在相同核心项集合的状态：
 - 如果已存在，则将当前状态中的 LR(1)项合并到已存在状态中。
 - 如果不存在，则将当前状态作为新的状态加入到 `coreToStateMap` 中。
 - 状态集合的更新：
 - 根据 `coreToStateMap` 中的结果，重新构建状态集合 `status`，确保合并后的状态集合中不包含重复的状态。
- LALR1 类中 `rebuildTable` 的算法：

- 构建新的操作表：
 - 使用 `unordered_map<pair<int, int>, vector<operation>, hash<pair<int, int>>> newTable` 来存储重新构建的操作表。
 - 操作表的键为 (状态编号, 符号编号) 对, 值为状态转移操作的向量。
- 遍历合并后的状态集合：
 - 对于每个状态, 遍历其包含的所有 LR(1) 项。
- 处理规约操作：
 - 如果 LR(1) 项已到达文法的末尾 (即点位置在产生式右部末尾), 则表示可以进行规约操作。
 - 构建规约操作并添加到操作表中, 确保不重复添加相同的规约操作。
- 处理移进操作：
 - 如果 LR(1) 项未到达文法的末尾, 则表示可以进行移进操作。
 - 获取 LR(1) 项的下一个符号, 构建移进操作, 并根据下一个符号找到对应的新状态。
 - 将移进操作添加到操作表中, 确保不重复添加相同的移进操作。
- 更新操作表：
 - 将构建好的 `newTable` 赋值给成员变量 `table`, 从而更新语法分析表。
- 详细步骤说明：
 - 遍历状态集合：
 - 对于每个状态 `current`, 获取其包含的所有 LR(1) 项。
 - 处理规约操作：
 - 如果 LR(1) 项的点位置在产生式右部的末尾, 表示可以进行规约操作。
 - 创建一个规约操作 `op`, 设置操作类型为规约, 并记录所用的文法编号 `item.grammar`。
 - 将规约操作 `op` 添加到操作表 `newTable` 中的适当位置。

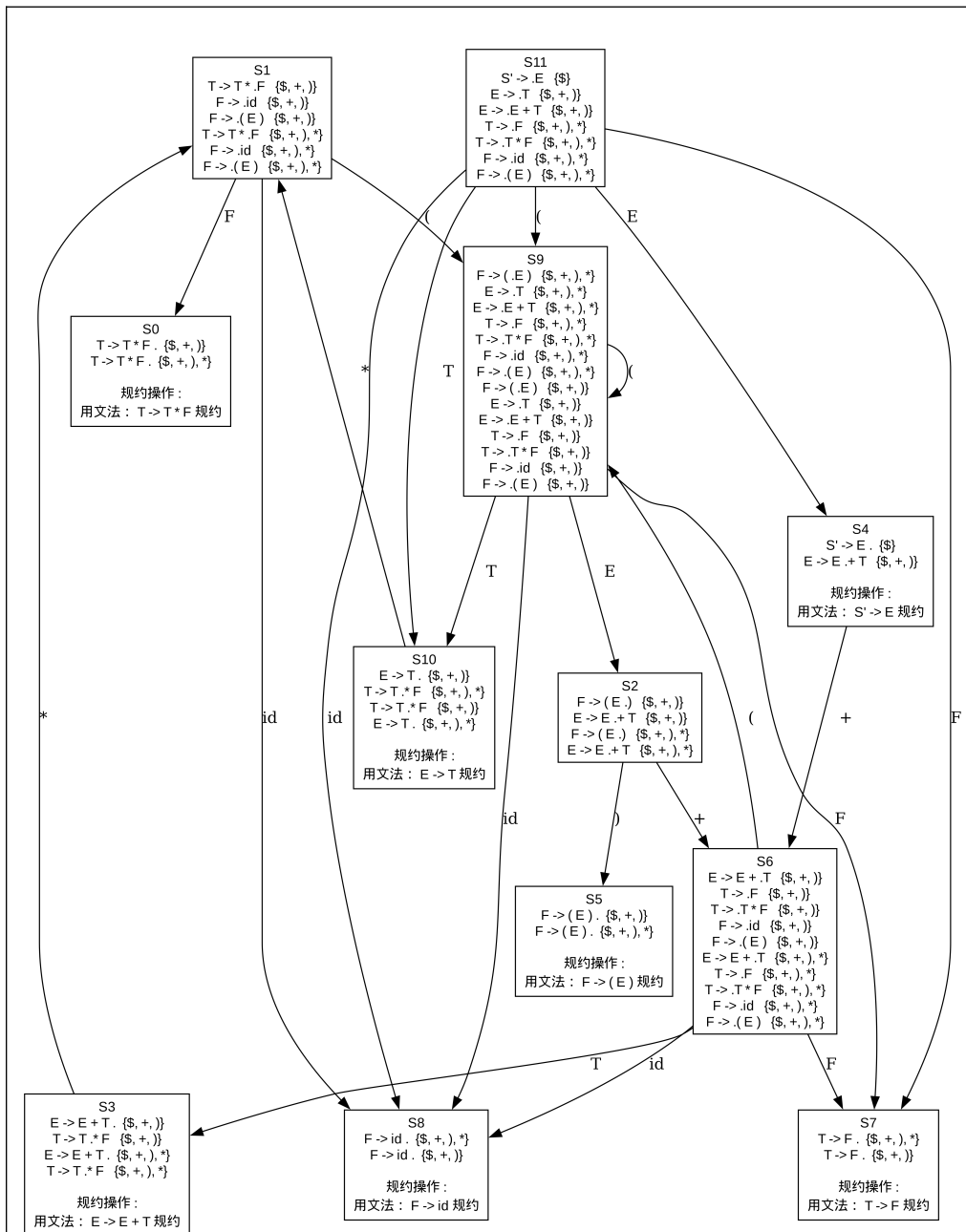
- 处理移进操作：
 - 如果 LR(1) 项的点位置不在产生式右部的末尾，表示可以进行移进操作。
 - 获取 LR(1) 项的下一个符号 nextSign，并创建一个移进操作 op，设置操作类型为移进，并记录目标状态的编号。
 - 将移进操作 op 添加到操作表 newTable 中的适当位置。
- 更新操作表：
 - 将构建好的 newTable 赋值给成员变量 table，从而更新语法分析表，确保 LALR(1) 分析器可以正确地分析输入文本。

5. 实验项目的测试

- 测试案例：
 - $E \rightarrow E + T$
 - $E \rightarrow T$
 - $T \rightarrow T * F$
 - $T \rightarrow F$
 - $F \rightarrow (E)$
 - $F \rightarrow id$
- 测试结果：

Symbol	First	Follow	Nullable
E	{(id }	{) + }	false
F	{(id }	{) * + }	false
T	{(id }	{) * + }	false

First 和 Follow 集合



LALR(1)DFA



界面示例：

四、实验总结（心得体会）

在完成这次实验后，我有以下几方面的心得体会：

1. 详细的需求分析是至关重要的。在开发过程中，明确每一个功能需求非常重要。提供一个文法输入编辑界面，并实现文法规则的保存和打开功能，能够帮助用户方便地管理和修改文法规则，减少了后续修改的复杂性和可能的错误。
2. First 集合与 Follow 集合的计算和展示。计算文法中各非终结符号的 First 集合和 Follow 集合是语法分析器设计中的关键步骤。通过提供窗口展示这些集合结果，用户可以清晰地了解到每个符号的终结符集合和跟随集合，这有助于分析语法分析器在处理文法时的行为。
3. LR(1)和 LALR(1) DFA 图的展示。实现文法对应的 LR(1)和 LALR(1) DFA 图的生成和展示，不仅是对算法正确性的验证，也是用户理解分析器工作原理的重要工具。使用图形或表格方式呈现这些图形数据，使用户可以直观地了解状态和状态之间的转移关系。
4. LALR(1)分析表的生成和展示。当文法为 LALR(1)文法时，生成并展示 LALR(1)分析表是应用程序的重要功能之一。分析表以表格形式展示，包括状态编号、输入符号和对应的操作（移进、规约或接受），帮助用户理解语法分析过程中的状态转移和操作行为。
5. 完善的软件文档和用户界面设计。编写完善的软件文档不仅有助于用户快速上手使用，还能帮助开发者在后续维护和扩展中保持代码的可维护性。为 Windows 界面设计应用程序，确保用户友好的操作体验，是提高软件应用价值的重要一环。

五、参考文献：

- [1] Kenneth C.Louden.编译原理及实践[M].机械工业出版社,2000-3-1.
- [2] 陆文周.Qt5 开发及实例（第 2 版）[M].第 2 版.电子工业出版社,2015-5-1.
- [3] Andrew W.Appel Maia Ginsburg.现代编译原理 C 语言描述[M].修订版.人民邮电出版社,2018-4-2.